

# 3-Tier Multi-Tenant Document Intelligence & LLM Parameter Simulator

---

This document provides a comprehensive, copy-pasteable **Master Prompt** to generate the codebase for this POC, followed by the **Architecture Blueprint** and **Deployment Guide** to showcase on your profile.

---

## Part 1: The Master Prompt (Copy & Paste)

---

You can copy the prompt below and feed it into any advanced AI coding assistant (like cursor, v0, Gemini, or Claude) to start generating the codebase.

Act as a Principal Software Engineer and Cloud Architect. Build a production-ready, clean, and modern web-based 3-Tier Proof of Concept (POC) application called "DocMind Studio".

The application is a Multi-Tenant Document Intelligence and LLM Parameter Simulator. It allows users to upload documents (PDF, TXT), configure dynamic system personas, customize queries, and simulate model outputs by adjusting LLM and chunking parameters (chunk size, overlap, temperature, top\_k, top\_p) with side-by-side comparisons.

### ### 1. High-Level Architecture & Tech Stack

- **\*\*Frontend (UI Tier):\*\*** React.js (Vite), Tailwind CSS, Lucide React (for icons), and Axios. Fast, responsive, and modern dashboard layout. Hostable on Netlify.
- **\*\*Backend (API Tier):\*\*** FastAPI (Python) or Express.js (Node.js/TypeScript). Clean RESTful API architecture, CORS enabled, and structured error handling.
- **\*\*Database (Data Tier):\*\*** Supabase (PostgreSQL) with pgvector. Must implement Multi-Tenancy using Row-Level Security (RLS) or tenant isolation columns.
- **\*\*LLM/AI Tier:\*\*** Integrated with major LLM APIs (OpenAI, Gemini, and Anthropic). Default to Gemini (using the Google Gen AI SDK) with a configuration screen to input API keys (secured per tenant).

### ### 2. Multi-Tenant Requirements

- Implement a simple multi-tenant system where each Tenant has its own:
  1. Profile & Settings (e.g., custom LLM preferences, Tenant name).
  2. Document library (stored securely with RLS).
  3. Generation history & Saved parameter simulation runs.
- Authentication can be simulated via a tenant-switch selector in the navbar for the POC, or implemented using Supabase Auth (with metadata fields mapping users to a tenant\_id).

### ### 3. Core Feature Requirements

#### #### A. Document Processing (Ingestion)

- Upload interface for `.txt` and `.pdf` files.
- Visual chunking simulator: When a user uploads a file, they can see a real-time preview of how the document is split into chunks based on current configurations (`chunk\_size` and `chunk\_overlap`).

#### #### B. Dynamic System Prompts (Personas)

- A configuration section to define the user's "Persona/Role" (e.g., "Senior Medical Auditor", "Legal Researcher", "Software QA Engineer").
- This Persona is dynamically injected as the `system` instruction in the LLM configuration.

#### #### C. LLM Parameter Simulation Playground

- Controls for adjusting:
  - **\*\*Model:\*\*** Dropdown (e.g., Gemini 1.5 Flash, Gemini 1.5 Pro, GPT-4o-mini, Claude 3.5 Sonnet).
  - **\*\*Chunking:\*\*** Chunk Size (characters/tokens) and Chunk Overlap.
  - **\*\*Sampling Params:\*\*** Temperature, Top\_P, Top\_K.
- **\*\*Side-by-Side Comparison:\*\*** Allow users to run the same query on the same document with *different* parameter sets (e.g., Config A: Chunk size 500, Temp 0.1 vs Config B: Chunk size 1500, Temp 0.8) and display the outputs next to each other to visually compare quality, source citations, and style.

#### #### D. Preset Queries

- Quick buttons to:
  1. "Summarize Document" (in a format specified by user: e.g., bullet points, executive brief).
  2. "Extract Key Entities" (e.g., dates, contract values, clauses).
  3. "Draft Follow-up Email" (using the document content as context).
  4. Custom free-form query.

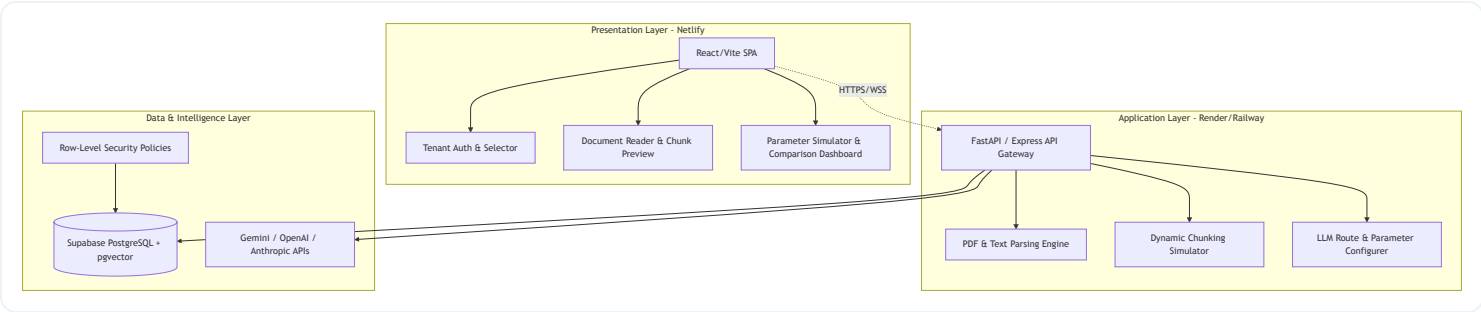
### ### 4. Implementation Deliverables

```
1. **Database Schema:** SQL script to create `tenants`, `documents`, `document_chunks`, and `generation_logs` tables with RLS policies mapping to `tenant_id`.
2. **Backend API:**
  - `/auth` endpoints for mock login / tenant switching.
  - `/documents` endpoints for uploading, deleting, and parsing text/PDFs.
  - `/simulate` endpoint that performs chunking, retrieves context (simulated vector search or simple regex-based chunk match for the POC), builds the prompt with the system persona, invokes the LLM API with target parameters, and saves the run details.
3. **Frontend SPA:**
  - A polished sidebar layout with tabs: Dashboard, Document Library, Persona Builder, and Simulator Playground.
  - Interactive UI displaying side-by-side simulation outputs and visual metrics (e.g., generation time, token counts, matching chunks).
  - Settings page for Tenant configurations (API keys, models).

Please provide the code structured in a clean monorepo folder layout: `/frontend` and `/backend`, with a detailed README.md including local run instructions and Netlify + Render/Railway deployment guides.
```

## Part 2: Technical Architecture Blueprint (For Profile Showcase)

When showcasing this project on your GitHub, Portfolio, or LinkedIn, use this architectural overview to explain the system's design.



### Technical Highlights

- 1. **Multi-Tenancy via PostgreSQL RLS:** Tenant data isolation is enforced at the database level. Each table has a `tenant_id` column. When a tenant queries the API, their JWT payload contains the `tenant_id`, and Supabase policies restrict access to only their rows.
- 2. **Chunking & Simulation Mechanics:** Real-time visualization of chunks helps developers and prompt engineers optimize retrieval strategy.
- 3. **Dynamic Orchestration:** The system constructs LLM requests dynamically:
- 4. System instruction: `You are acting as a [User Persona]. [Default Instructions]`
- 5. User prompt: `Analyze the following context chunks and answer the question: [Query]`
- 6. Params: `temperature`, `top_k`, `top_p` are forwarded directly to the selected model SDK.

## Part 3: Step-by-Step Deployment Strategy

Here is how you can deploy this architecture for free/cheap to showcase a fully functional demo:

## 1. Database & Auth (Supabase)

- Create a free project on [Supabase](#).
- Turn on Row Level Security (RLS) on all tables.
- Define a tenant-level access policy: `sql CREATE POLICY tenant_isolation_policy ON documents FOR ALL TO authenticated USING (tenant_id = auth.jwt() ->> 'tenant_id');`

## 2. Frontend (Netlify)

- Netlify connects directly to your GitHub repository.
- Deploy the `/frontend` subfolder as a Single Page Application (SPA).
- **Netlify configuration ( `netlify.toml` ):** `toml [[redirects]] from = "/*" to = "/index.html" status = 200`
- Configure environment variables:
- `VITE_API_BASE_URL` : Pointing to your deployed backend API.

## 3. Backend (Render / Railway / Fly.io)

- Deploy your backend web service to Render, Railway, or Fly.io.
- Add environment variables:
- `DATABASE_URL` : Your Supabase connection string.
- `GEMINI_API_KEY` , `OPENAI_API_KEY` : Default API keys (or allow users to supply theirs dynamically).